Computational Economics

Lecture: Q Functions, Optimality, and Value Function Iteration

Yaolang Zhong yaolang.zhong@e.u-tokyo.ac.jp
Graduate School of Economics, The University of Tokyo

Lecture Overview

- **From a Fixed Policy to Policy Improvement**
  - State values, state-action values, and the maze example
  - Greedy choices and the policy improvement theorem
- **Optimality and Policy Iteration**
  - Bellman optimality and the one-step deviation principle
  - Exact policy evaluation and improvement
- **Value Function Iteration**
  - Synchronous updates and convergence
  - Stopping errors and policy recovery

Recap: Computing the Value Function of a Given Policy

Consider a specific deterministic policy σ.

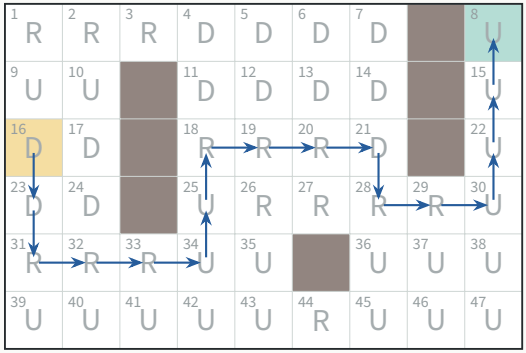


Figure 1: Actions and path under σ.

- U: up; D: down;
  L: left; R: right.
- Each move gives -1, including entry into cell 8.
- Set $\beta = 1$ for simplicity.
- Following this policy reaches cell 8 from every free cell.

Recap: Computing the Value Function of a Given Policy

With $\beta = 1$, compute values backward from cell 8, following the actions of σ.

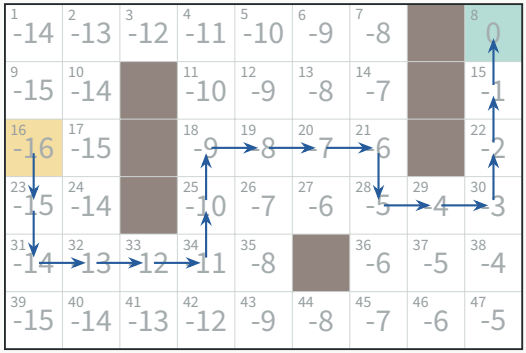


Figure 2: Values and path under σ.

$V^\sigma(8) = 0$. For every other cell,

$$V^\sigma(s) = -1 + V^\sigma(s').$$

$$V^\sigma(15) = -1 + 0 = -1,$$
$$V^\sigma(22) = -1 + (-1) = -2,$$
$$V^\sigma(30) = -1 + (-2) = -3.$$

Continuing backward gives $V^\sigma(16) = -16$.

Each value is minus the number of moves to cell 8 under σ.

From State Values to Action Values

The **state value function** $V^\sigma(s)$: follow the deterministic policy σ now and afterward.

$$V^\sigma(s) = \underbrace{r\big(s, \sigma(s)\big)}_{\text{Immediate reward}} + \underbrace{\beta\, \mathbb{E}[V^\sigma(s') \mid s,\ a = \sigma(s)]}_{\text{Discounted continuation value}}.$$

The **state-action value function** $Q^\sigma(s, a)$: choose a now, then follow σ.

$$Q^\sigma(s, a) = \underbrace{r(s, a)}_{\text{Immediate reward}} + \underbrace{\beta\, \mathbb{E}[V^\sigma(s') \mid s, a]}_{\text{Discounted continuation value}}.$$

The first action a need not be the action selected by σ.

If the first action also follows σ, then

$$\text{Deterministic:} \quad V^\sigma(s) = Q^\sigma(s, \sigma(s)),$$

$$\text{Randomized:} \quad V^\sigma(s) = \sum_{a \in \mathcal{A}(s)} \Pr\{\sigma(s) = a\}\, Q^\sigma(s, a).$$

Example: Comparing First Actions

At cell 25, $\sigma(25) = \mathrm{U}$. Compare U and R, then follow σ.

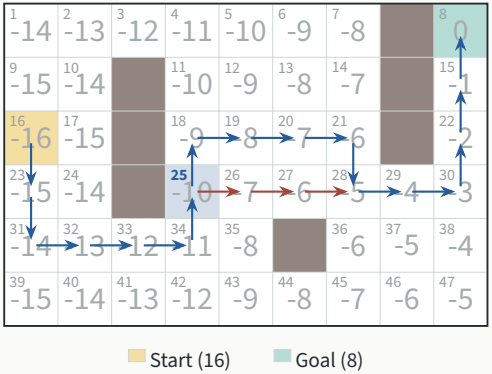


Figure 3: Policy path and shortcut from cell 25.

Up to cell 18:

$$Q^{\sigma}(25, \mathrm{U}) = -1 - 9 = -10.$$

Right to cell 26:

$$Q^{\sigma}(25, \mathrm{R}) = -1 - 7 = -8.$$

The policy chooses U, so $V^{\sigma}(25) = -10$.
Choosing R first gives a higher return.

Greedy Policy Improvement

For a given policy σ, let V^σ and Q^σ denote its state and state–action value functions. At a particular state s, a greedy **policy improvement step** defines the new policy $\widetilde{\sigma}$ by

$$\widetilde{a} \in \arg\max_{a \in \mathcal{A}(s)} Q^\sigma(s,a), \qquad \widetilde{\sigma}(x) = \begin{cases} \widetilde{a}, & x = s, \\ \sigma(x), & x \neq s. \end{cases}$$

i.e., the new policy uses $\widetilde{a}$ at s and keeps σ at every other state.

Policy improvement theorem. More generally, let σ and $\widetilde{\sigma}$ be any two stationary deterministic policies. Assume $|r(s,a)| \leq \bar{r} < \infty$ for all feasible (s,a), and $0 < \beta < 1$. If

$$Q^\sigma(s, \widetilde{\sigma}(s)) \geq V^\sigma(s) \qquad \forall s \in \mathcal{S},$$

then

$$V^{\widetilde{\sigma}}(s) \geq V^\sigma(s) \qquad \forall s \in \mathcal{S}.$$

If, at every state s, taking $\widetilde{\sigma}(s)$ once and then following σ is no worse than following σ throughout, then following $\widetilde{\sigma}$ at every step is also no worse.

Policy Improvement: Proof

The **Bellman operator for the fixed policy** $\widetilde{\sigma}$ maps an input continuation-value function V to a new function $T_{\widetilde{\sigma}}V$:

$$(T_{\widetilde{\sigma}}V)(x) = r(x, \widetilde{\sigma}(x)) + \beta\mathbb{E}[V(x') \mid x, a = \widetilde{\sigma}(x)].$$

Starting from V^{σ}, one application follows $\widetilde{\sigma}$ for one step, then σ afterward. The theorem's condition gives

$$(T_{\widetilde{\sigma}}V^{\sigma})(x) = Q^{\sigma}(x, \widetilde{\sigma}(x)) \geq V^{\sigma}(x), \qquad \forall x.$$

Monotonicity means preserving the ordering of two candidate value functions V_1, V_2:

$$V_1(x) \leq V_2(x)\ \forall x \quad \Longrightarrow \quad (T_{\widetilde{\sigma}}V_1)(x) \leq (T_{\widetilde{\sigma}}V_2)(x)\ \forall x.$$

Higher continuation values cannot lower the updated value. Applying this repeatedly gives

$$V^{\sigma} \leq T_{\widetilde{\sigma}}V^{\sigma} \leq T_{\widetilde{\sigma}}^{2}V^{\sigma} \leq \cdots.$$

$T_{\widetilde{\sigma}}^{n}V^{\sigma}$ is the value of following $\widetilde{\sigma}$ for n steps, then σ. The remaining continuation has vanishing weight β^{n}, so

$$\lim_{n\to\infty}(T_{\widetilde{\sigma}}^{n}V^{\sigma})(x) = V^{\widetilde{\sigma}}(x) \geq V^{\sigma}(x), \qquad \forall x.$$

The Principle of Optimality

Define the **optimal values** over feasible policies:

$$V^*(s) = \max_\sigma V^\sigma(s), \qquad Q^*(s,a) = \max_\sigma Q^\sigma(s,a).$$

Principle of optimality: an optimal policy must have an optimal continuation from every state reached with positive probability.

$$Q^*(s,a) = r(s,a) + \beta \mathbb{E}[V^*(s') \mid s,a],$$
$$V^*(s) = \max_{a \in \mathcal{A}(s)} Q^*(s,a).$$

Combining them gives the **Bellman optimality equation**:

$$V^*(s) = \max_{a \in \mathcal{A}(s)} \{r(s,a) + \beta \mathbb{E}[V^*(s') \mid s,a]\}.$$

Optimality as a One-Step Look Ahead

$$V^*(s) = \max_a Q^*(s, a)$$

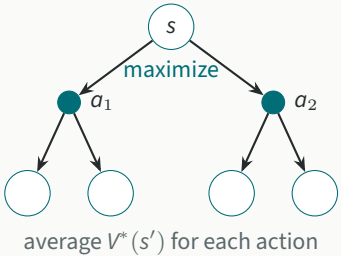


Figure 4: Choose now, average outcomes.

$$Q^*(s, a) = r(s, a) + \beta \mathbb{E}[\max_{a'} Q^*(s', a') \mid s, a]$$

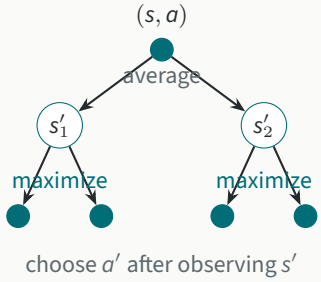


Figure 5: Fix now, choose at the next state.

Open circles: states. Filled circles: state–action pairs. Adapted from Sutton and Barto (2018), Figure 3.4; branches show possible next states.

The Bellman Optimality Operator

Replace the optimal continuation by a candidate V. The **Bellman optimality operator** is

$$(TV)(s) = \max_{a \in \mathcal{A}(s)} \left\{ r(s,a) + \beta \sum_{s' \in \mathcal{S}} P(s' \mid s,a) V(s') \right\}.$$

- $T_\sigma V$ uses the prescribed action $\sigma(s)$; TV chooses the best action for continuation V.
- A policy greedy with respect to V attains this maximum: $T_\sigma V = TV$.
- The optimal value satisfies $TV^* = V^*$: it is a **fixed point** of T.

Contraction and Uniqueness

For two candidate value functions V_1, V_2, use the maximum distance

$$\|V_1 - V_2\|_\infty = \max_s |V_1(s) - V_2(s)|.$$

For the same state and action,

$$|\beta \mathbb{E}[V_1(s') - V_2(s') \mid s, a]| \leq \beta \|V_1 - V_2\|_\infty.$$

Taking maxima cannot enlarge this bound, since

$$|\max_a x_a - \max_a y_a| \leq \max_a |x_a - y_a|.$$

Therefore T is a **contraction**:

$$\|TV_1 - TV_2\|_\infty \leq \beta \|V_1 - V_2\|_\infty.$$

The contraction mapping theorem gives a unique fixed point, V^*, and convergence to it from any initial value vector. The same bound holds for each fixed-policy operator T_σ.

When Is a Policy Optimal?

The **one-step deviation principle** gives a test using the policy's own value:

$$\sigma(s) \in \arg\max_{a \in \mathcal{A}(s)} Q^{\sigma}(s,a) \quad \forall s \quad \Longleftrightarrow \quad V^{\sigma} = V^{*}.$$

- **Sufficiency:** if no first-action change improves the return at any state, then

  $$V^{\sigma} = T_{\sigma} V^{\sigma} = T V^{\sigma}.$$

  The Bellman fixed point is unique, so $V^{\sigma} = V^{*}$.

- **Necessity:** if $V^{\sigma} = V^{*}$, then

  $$T_{\sigma} V^{*} = V^{*} = T V^{*},$$

  so σ attains the maximum at every state.

Policy Iteration

Start from any feasible policy σ_0. Repeat:

1. **Policy evaluation:** solve exactly for the current policy's value,

$$V^{\sigma_k} = T_{\sigma_k} V^{\sigma_k}.$$

2. **Policy improvement:** choose

$$\sigma_{k+1}(s) \in \arg\max_{a \in \mathcal{A}(s)} Q^{\sigma_k}(s, a), \qquad \forall s.$$

   Keep the current action whenever it is a maximizer.

3. If $\sigma_{k+1} = \sigma_k$, stop: the policy is optimal. Otherwise evaluate the new policy and repeat.

$$V^{\sigma_0} \leq V^{\sigma_1} \leq V^{\sigma_2} \leq \cdots \leq V^*.$$

Each nonoptimal policy improves strictly at some state. With finitely many policies, this process terminates at an optimum.

Value Function Iteration

VFI updates a value estimate V_k, without evaluating a policy to completion.

1. Initialize V_0, for example at zero.
2. Using the old values, compute the scores and the new values:

$$Q_k(s,a) = r(s,a) + \beta \mathbb{E}[V_k(s') \mid s,a],$$
$$V_{k+1}(s) = \max_{a \in \mathcal{A}(s)} Q_k(s,a) = (TV_k)(s).$$

3. Repeat these updates until the required accuracy is reached.

A **sweep** updates all states using the same old V_k; store the results in a separate array.

Contraction gives

$$\|V_k - V^*\|_\infty \leq \beta^k \|V_0 - V^*\|_\infty \longrightarrow 0.$$

The estimates V_k and Q_k need not be the values of a particular policy.

Stopping VFI and Recovering a Policy

After computing $V_{k+1} = TV_k$, measure the **Bellman residual**:

$$d_k = \|TV_k - V_k\|_\infty = \|V_{k+1} - V_k\|_\infty.$$

Contraction implies

$$\|V_k - V^*\|_\infty \le d_k + \beta\|V_k - V^*\|_\infty, \quad \text{so} \quad \|V_k - V^*\|_\infty \le \frac{d_k}{1-\beta}.$$

- For value error at most ε, stop when $d_k \le (1-\beta)\varepsilon$ and return V_k.
- Recover a policy from the scores computed using this returned value:

$$\widehat{\sigma}(s) \in \arg\max_{a \in \mathcal{A}(s)} Q_k(s,a).$$

- Evaluate $V^{\widehat{\sigma}}$ to check the policy's achieved return. Intermediate greedy policies in VFI need not improve monotonically.

References and Further Reading

1. **Richard S. Sutton and Andrew G. Barto (2018), Reinforcement Learning: An Introduction. 2nd ed. MIT Press. [Book]**
   - Chapter 3: Finite Markov Decision Processes
   - Chapter 4: Dynamic Programming
2. **Thomas J. Sargent and John Stachurski, QuantEcon. [Online lectures]**
   - Discrete State Dynamic Programming — The Bellman Equation and the Principle of Optimality
   - Discrete State Dynamic Programming — Policy Function Iteration
   - Discrete State Dynamic Programming — Value Function Iteration